.cell-output-stdout code { word-break: break-wor !important; white-space: pre-wrap !important; } </
style>

Nettoyer et structurer l'information dans les données textuelles

Lino Galiana
Version du 20 octobre 2026

Les corpus textuels étant des objets de très grande dimension où le ratio signal/bruit est faible, il est nécessaire de mettre en oeuvre une série d'étapes de nettoyage de texte. Ce chapitre va explorer quelques méthodes classiques de nettoyage en s'appuyant sur des corpus littéraires: le *Comte de Monte Cristo* d'Alexandre Dumas et des auteurs fantastiques anglo-saxons (Lovecraft, Poe, Shelley).

⚠️ **PDF généré automatiquement.** Ce document est produit à partir du site du cours et peut présenter des problèmes de mise en forme. Pour une version à jour et adaptée à la lecture, consultez la page du cours : pythonds.linogaliana.fr.

Table des matières

1 Introduction **2**
1.1 Rappel 2
1.2 Objectif du chapitre 3
1.3 Méthode 3
2 Bases d'exemple **4**
2.1 Le *Comte de Monte Cristo* 4
2.2 Le corpus anglo-saxon 5
3 Premières analyses de fréquence **6**
3.1 Exploration ponctuelle 6
3.2 Transformation d'un texte en *tokens* 8
3.3 Le nuage de mot: une première analyse généralisée 9
3.4 Aparté: la loi de Zipf 15
4 Nettoyage de textes **18**
4.1 Retirer les *stop words* 18
4.2 Racinisation et lemmatisation 22
4.3 Limite 23
Informations additionnelles 24
Bibliographie **25**

1 Introduction

1.1 Rappel

Comme évoqué dans l'introduction de cette partie sur le traitement automatique du langage, l'objectif principal des techniques que nous allons explorer est la représentation synthétique du langage.

Le *natural language processing* (NLP) ou *traitement automatisé du langage* (TAL) en Français, vise à extraire de l'information de textes à partir d'une analyse statistique du contenu. Cette définition permet d'inclure de nombreux champs d'applications au sein du NLP (traduction, analyse de sentiment, recommandation, surveillance, etc.).

Cette approche implique de transformer un texte, qui est une information compréhensible par un humain, en un nombre, information appropriée pour un ordinateur dans le cadre d'une approche statistique ou algorithmique.

Transformer une information textuelle en valeurs numériques propres à une analyse statistique n'est pas une tâche évidente. Les données textuelles sont **non structurées** puisque l'information cherchée, qui est propre à chaque analyse, est perdue au milieu d'une grande masse d'informations qui doit, de plus, être interprétée dans un certain contexte (un même mot ou une phrase n'ayant pas la même signification selon le contexte).

Si cette tâche n'était pas assez difficile comme ça, on peut ajouter d'autres difficultés propres à l'analyse textuelle car ces données sont :

- **bruitées** : ortographe, fautes de frappe…
- **changeantes** : la langue évolue avec de nouveaux mots, sens…
- **complexes** : structures variables, accords…
- **ambiguës** : synonymie, polysémie, sens caché…
- **propres à chaque langue** : il n'existe pas de règle de passage unique entre deux langues
- de **grande dimension** : des combinaisons infinies de séquences de mots

1.2 Objectif du chapitre

Dans ce chapitre, nous allons nous restreindre aux méthodes fréquentistes dans le paradigme *bag of words*. Celles-ci sont un peu *old school* par rapport aux approches plus raffinées que nous évoquerons ultérieurement. Néanmoins, les présenter nous permettra d'évoquer un certain nombre d'enjeux typiques des données textuelles qui restent centraux dans le NLP moderne.

Le principal enseignement à retenir de cette partie est que les données textuelles étant à très haute dimension - le langage étant un objet riche - nous avons besoin de méthodes pour réduire le bruit de nos corpus textuels afin de mieux prendre en compte le signal en leur sein.

Cette partie est une introduction s'appuyant sur quelques ouvrages classiques de la littérature française ou anglo-saxonne. Seront notamment présentées quelques librairies faisant parti de la boite à outil minimale des *data scientists*: `NLTK` et `SpaCy`. Les chapitres suivants permettront de se focaliser sur la modélisation du langage.

> ℹ La librairie SpaCy
>
> `NTLK` est la librairie historique d'analyse textuelle en `Python`. Elle existe depuis les années 1990. L'utilisation industrielle du NLP dans le monde de la *data science* est néanmoins plus récente et doit beaucoup à la collecte accrue de données non structurées par les réseaux sociaux. Cela a amené à un renouvellement du champ du NLP, tant dans le monde de la recherche que dans sa mise en application dans l'industrie de la donnée.
>
> Le *package* `spaCy` est l'un des packages qui a permis cette industrialisation des méthodes de NLP. Conçu autour du concept de *pipelines* de données, il est beaucoup plus pratique à mettre en oeuvre pour une chaîne de traitement de données textuelles mettant en oeuvre plusieurs étapes de transformation des données.

1.3 Méthode

L'analyse textuelle vise à transformer le texte en données numériques manipulables. Pour cela il est nécessaire de se fixer une unité sémantique minimale. Cette unité textuelle peut être le mot ou encore une séquence de n mots (un *ngram*) ou encore une chaîne de caractères (e.g. la ponctuation peut être signifiante). On parle de **token**.

On peut ensuite utiliser diverses techniques (*clustering*, classification supervisée) suivant l'objectif poursuivi pour exploiter l'information transformée. Mais les étapes de nettoyage de texte sont indispensables. Sinon un algorithme sera incapable de détecter une information pertinente dans l'infini des possibles.

Les *packages* suivants seront utiles au cours de ce chapitre:

```
!pip install pywaffle
!pip install spacy
!pip install plotnine
!pip install great_tables
!pip install wordcloud
```

2 Bases d'exemple

2.1 Le *Comte de Monte Cristo*

La base d'exemple est le *Comte de Monte Cristo* d'Alexandre Dumas. Il est disponible gratuitement sur le site http://www.gutenberg.org *(Project Gutemberg)* comme des milliers d'autres livres du domaine public.

La manière la plus simple de le récupérer est de télécharger avec le *package* `request` le fichier texte et le retravailler légèrement pour ne conserver que le corpus du livre :

```
import requests
import re

url = "https://www.gutenberg.org/files/17989/17989-0.txt"
response = requests.get(url)
response.encoding = 'utf-8'  # Assure le bon décodage
raw = response.text

dumas = (
  raw
  .split("*** START OF THE PROJECT GUTENBERG EBOOK 17989 ***")[1]
  .split("*** END OF THE PROJECT GUTENBERG EBOOK 17989 ***")[0]
)


def clean_text(text):
    text = text.lower() # mettre les mots en minuscule
    text = " ".join(text.split())
    return text

dumas = clean_text(dumas)

dumas[10000:10500]
```

Ligne 13

On extrait de manière un petit peu simpliste le contenu de l'ouvrage

```
" mes yeux. --vous avez donc vu l'empereur aussi? --il est entré chez le maréchal pendant que
j'y étais. --et vous lui avez parlé? --c'est-à-dire que c'est lui qui m'a parlé, monsieur, dit
dantès en souriant. --et que vous a-t-il dit? --il m'a fait des questions sur le bâtiment, sur
```

```
l'époque de son départ pour marseille, sur la route qu'il avait suivie et sur la cargaison qu'il
portait. je crois que s'il eût été vide, et que j'en eusse été le maître, son intention eût été
de l'acheter; mais je lu"
```

2.2 Le corpus anglo-saxon

Nous allons utiliser une base anglo-saxonne présentant trois auteurs de la littérature fantastique:

- Edgar Allan Poe, (*EAP*) ;
- HP Lovecraft (*HPL*) ;
- Mary Wollstonecraft Shelley (*MWS*).

Les données sont disponibles sur un CSV mis à disposition sur `Github`. L'URL pour les récupérer directement est https://github.com/GU4243-ADS/spring2018-project1-ginnyqg/raw/master/data/spooky.csv.

Le fait d'avoir un corpus confrontant plusieurs auteurs nous permettra de comprendre la manière dont les nettoyages de données textuelles favorisent les analyses comparatives.

Nous pouvons utiliser le code suivant pour lire et préparer ces données:

```
import pandas as pd

url='https://github.com/GU4243-ADS/spring2018-project1-ginnyqg/raw/master/data/spooky.csv'
#1. Import des données
horror = pd.read_csv(url,encoding='latin-1')
#2. Majuscules aux noms des colonnes
horror.columns = horror.columns.str.capitalize()
#3. Retirer le prefixe id
horror['ID'] = horror['Id'].str.replace("id","")
horror = horror.set_index('Id')
```

```

```

Le jeu de données met ainsi en regard un auteur avec une phrase qu'il a écrite :

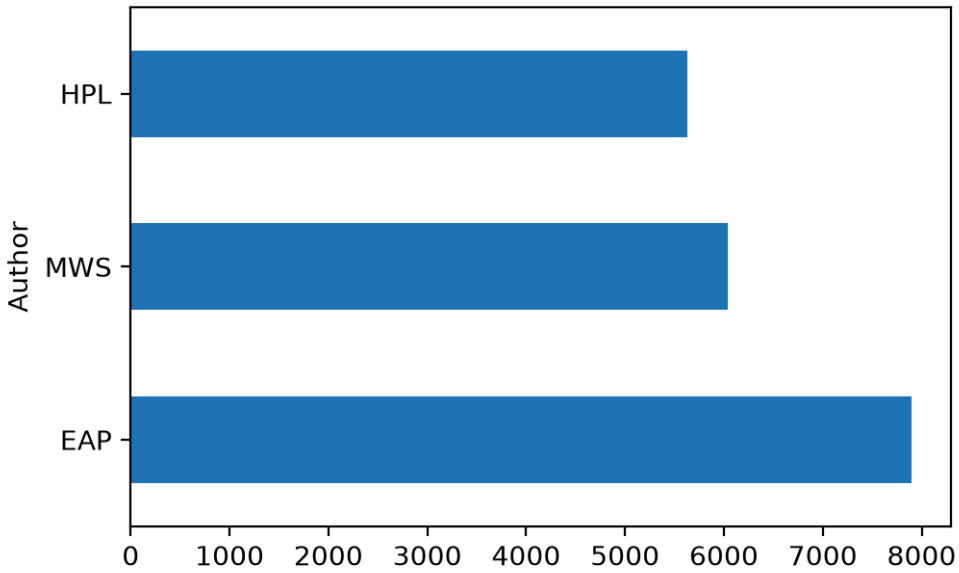
```
horror.head()
```

Id	Text	Author	ID
id26305	This process, however, afforded me no means of…	EAP	26305
id17569	It never once occurred to me that the fumbling…	HPL	17569
id11008	In his left hand was a gold snuff box, from wh…	EAP	11008
id27763	How lovely is spring As we looked from Windsor…	MWS	27763
id12958	Finding nothing else, not even gold, the Super…	HPL	12958

On peut se rendre compte que les extraits des 3 auteurs ne sont pas forcément équilibrés dans le jeu de données. Si on utilise ultérieurement ce corpus pour de la modélisation, il sera nécessaire de tenir compte de ce déséquilibre.

```
(
  horror
  .value_counts('Author')
```

```
  .plot(kind = "barh")
)
```



3 Premières analyses de fréquence

L'approche usuelle en statistique, qui consiste à faire une analyse descriptive avant de mettre en oeuvre une modélisation, s'applique également à l'analyse de données textuelles. La fouille de documents textuels implique ainsi, en premier lieu, une analyse statistique afin de déterminer la structure du corpus.

Avant de s'adonner à une analyse systématique du champ lexical de chaque auteur, on va se focaliser dans un premier temps sur un unique mot, le mot *fear*.

3.1 Exploration ponctuelle

Tip

L'exercice ci-dessous présente une représentation graphique nommée *waffle chart*. Il s'agit d'une approche préférable aux camemberts (*pie chart*) qui sont des graphiques manipulables car l'oeil humain se laisse facilement berner par cette représentation graphique qui ne respecte pas les proportions.

Exercice 1 : Fréquence d'un mot

Dans un premier temps, nous allons nous concentrer sur notre corpus anglo-saxon (`horror`)

1. Compter le nombre de phrases, pour chaque auteur, où apparaît le mot `fear`.
2. Utiliser `pywaffle` pour obtenir les graphiques ci-dessous qui résument de manière synthétique le nombre d'occurrences du mot « *fear* » par auteur.
3. Refaire l'analyse avec le mot « *horror* ».

Le comptage obtenu devrait être le suivant

	wordtoplot
Author	
EAP	70
HPL	160
MWS	211

Ceci permet d'obtenir le *waffle chart* suivant :

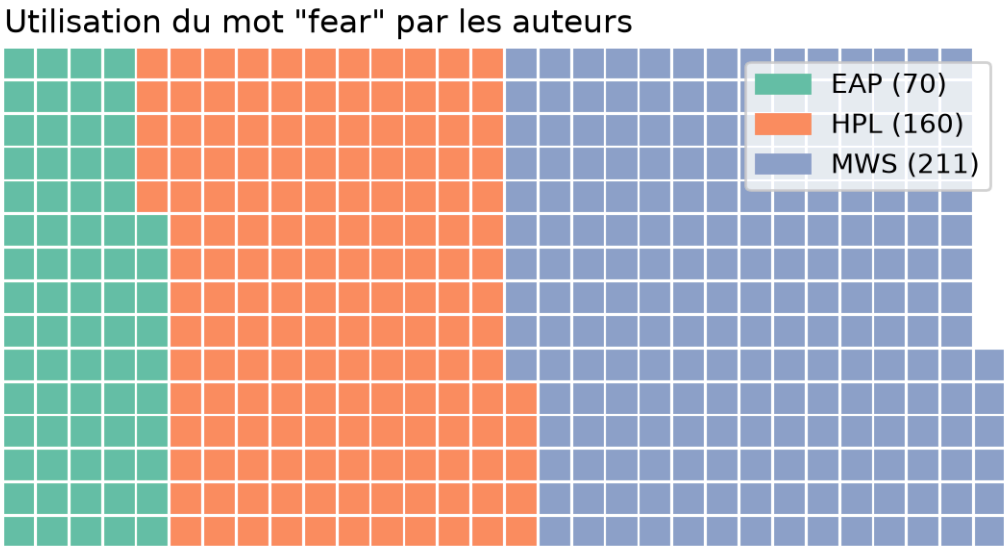


Figure 1. – Répartition du terme fear dans le corpus de nos trois auteurs

On remarque ainsi de manière très intuitive le déséquilibre de notre jeu de données lorsqu'on se focalise sur le terme *« peur »* où Mary Shelley représente près de 50% des observations.

Si on reproduit cette analyse avec le terme *« horror »*, on retrouve la figure suivante:

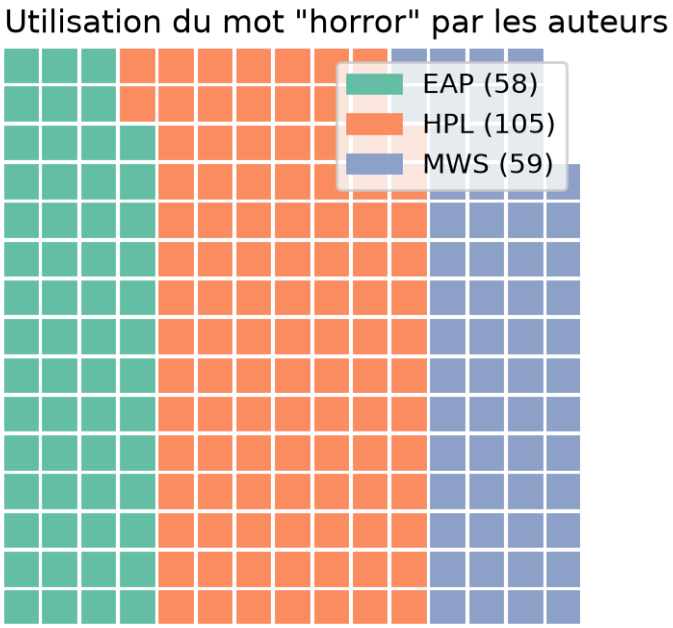


Figure 2. – Répartition du terme horror dans le corpus de nos trois auteurs

3.2 Transformation d'un texte en *tokens*

Dans l'exercice précédent, nous faisions une recherche ponctuelle, qui ne passe pas vraiment à l'échelle. Pour généraliser cette approche, on découpe généralement un corpus en unités sémantiques indépendantes : les *tokens*.

> Tip
>
> Nous allons avoir besoin d'importer un certain nombre de corpus prêts à l'emploi pour utiliser les librairies `NTLK` ou `SpaCy`. Les instructions ci-dessous permettront de récupérer toutes ces ressources

Pour récupérer tous nos corpus `NLTK` prêts à l'emploi, nous faisons

```
import nltk
nltk.download('stopwords')
nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('genesis')
nltk.download('wordnet')
nltk.download('omw-1.4')
```

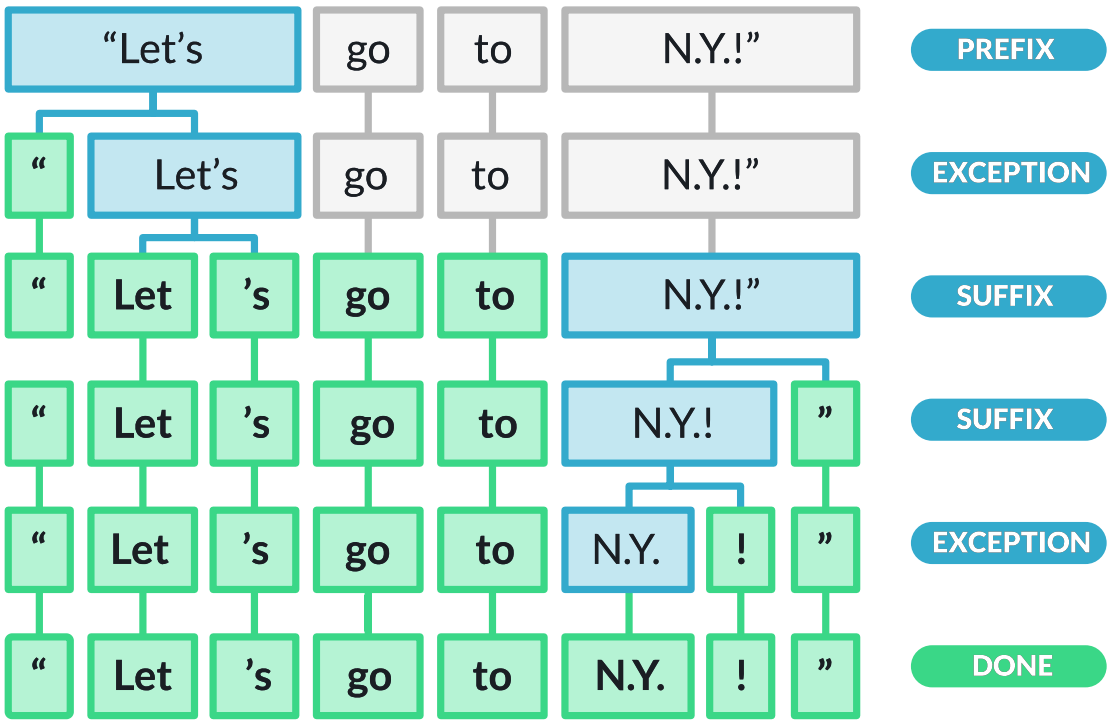
En ce qui concerne `SpaCy`, il est nécessaire d'utiliser la ligne de commande:

```
!python -m spacy download fr_core_news_sm
!python -m spacy download en_core_web_sm
```

Plutôt que d'implémenter soi-même un *tokenizer* inefficace, il est plus approprié d'en appliquer un issu d'une librairie spécialisée. Historiquement, le plus simple était de prendre le *tokenizer* de `NLTK`, la librairie historique de *text mining* en `Python`:

```
from nltk.tokenize import word_tokenize
word_tokenize(dumas[10000:10500])
```

Comme on le voit, cette librairie ne fait pas les choses dans le détail et a quelques incohérences: `j'y étais` est séparé en 4 sèmes (`['j', "'", 'y', 'étais']`) là où `l'acheter` reste un unique sème. `NLTK` est en effet une librairie anglo-saxonne et l'algorithme de séparation n'est pas toujours adapté aux règles grammaticales françaises. Il vaut mieux dans ce cas privilégier `SpaCy`, la librairie plus récente pour faire ce type de tâche. En plus d'être très bien documentée, elle est mieux adaptée pour les langues non anglo-saxonnes. En l'occurrence, comme le montre l'exemple de la documentation sur les *tokenizers*, l'algorithme de séparation présente un certain raffinement



Celui-ci peut être appliqué de cette manière:

```
import spacy
from spacy.tokenizer import Tokenizer
nlp = spacy.load("fr_core_news_sm")
doc = nlp(dumas[10000:10500])

text_tokenized = []
for token in doc:
    text_tokenized += [token.text]

", ".join(text_tokenized)
```

```
" , mes, yeux, ., --vous, avez, donc, vu, l', empereur, aussi, ?, --il, est, entré, chez, le,
maréchal, pendant, que, j', y, étais, ., --et, vous, lui, avez, parlé, ?, --c', est, -, à, -,
dire, que, c', est, lui, qui, m', a, parlé, ,, monsieur, ,, dit, dantès, en, souriant, ., --et,
que, vous, a, -t, -il, dit, ?, --il, m', a, fait, des, questions, sur, le, bâtiment, ,, sur, l',
époque, de, son, départ, pour, marseille, ,, sur, la, route, qu', il, avait, suivie, et, sur,
la, cargaison, qu', il, portait, ., je, crois, que, s', il, eût, été, vide, ,, et, que, j', en,
eusse, été, le, maître, ,, son, intention, eût, été, de, l', acheter, ;, mais, je, lu"
```

Comme on peut le voir, il reste encore beaucoup d'éléments polluants notre structuration de corpus, à commencer par la ponctuation. Nous allons néanmoins pouvoir facilement retirer ceux-ci ultérieurement, comme nous le verrons.

3.3 Le nuage de mot: une première analyse généralisée

A ce stade, nous n'avons encore aucune appréhension de la structure de notre corpus : nombre de mots, mots les plus représentés, etc.

Pour se faire une idée de la structure de notre corpus, on peut commencer par compter la distribution des mots dans l'oeuvre de Dumas. Commençons par le début de l'oeuvre, à savoir les 30 000 premiers mots et comptons les mots uniques :

```
from collections import Counter

doc = nlp(dumas[:30000])

# Extract tokens, convert to lowercase and filter out punctuation and spaces
tokens = [token.text.lower() for token in doc if not token.is_punct and not token.is_space]

# Count the frequency of each token
token_counts = Counter(tokens)
```

Nous avons déjà de nombreux mots différents dans le début de l'oeuvre.

```
len(token_counts)
```

```
1401
```

Nous voyons la haute dimensionnalité du corpus puisque nous avons près de 1500 mots différents sur les 30 000 premiers mots de l'oeuvre de Dumas.

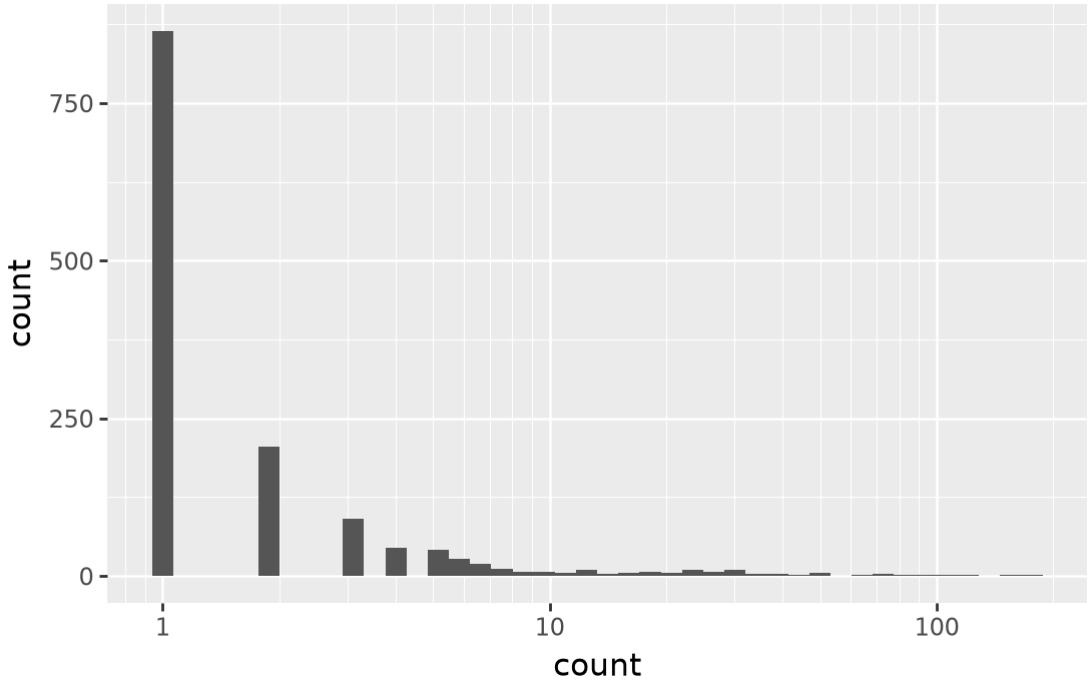
```
token_count_all = list(token_counts.items())

# Create a DataFrame from the list of tuples
token_count_all = pd.DataFrame(token_count_all, columns=['word', 'count'])
```

Si on regarde la distribution de la fréquence des mots, exercice que nous prolongerons ultérieurement en évoquant la loi de Zipf, nous pouvons voir que de nombreux mots sont unique (près de la moitié des mots), que la densité de fréquence descend vite et qu'il faudrait se concentrer un peu plus sur la queue de distribution que ne le permet la figure suivante :

```
from plotnine import *
(
  ggplot(token_count_all) +
  geom_histogram(aes(x = "count")) +
  scale_x_log10()
)
```

```
/__w/python-datascientist/python-datascientist/.venv/lib/python3.13/site-packages/plotnine/
stats/stat_bin.py:112: PlotnineWarning: 'stat_bin()' using 'bins = 42'. Pick better value with
'binwidth'.
```



Maintenant, si on regarde les 25 mots les plus fréquents, on peut voir que ceux-ci ne sont pas très intéressants pour analyser le sens de notre document :

```
# Sort the tokens by frequency in descending order
sorted_token_counts = token_counts.most_common(25)
sorted_token_counts = pd.DataFrame(sorted_token_counts, columns=['word', 'count'])
```

Mot	Nombre d'occurrences	
de	176	
le	149	
et	124	
à	116	
l'	102	
que	100	

Nombre d'apparitions sur les 30 000 premiers caractères du Comte de Monte Cristo

Mot	Nombre d'occurrences	
vous	88	
la	83	
il	81	
un	77	
je	73	
en	70	
est	61	
qui	53	
dantès	53	
d'	50	
dit	50	
les	49	
du	46	
a	41	

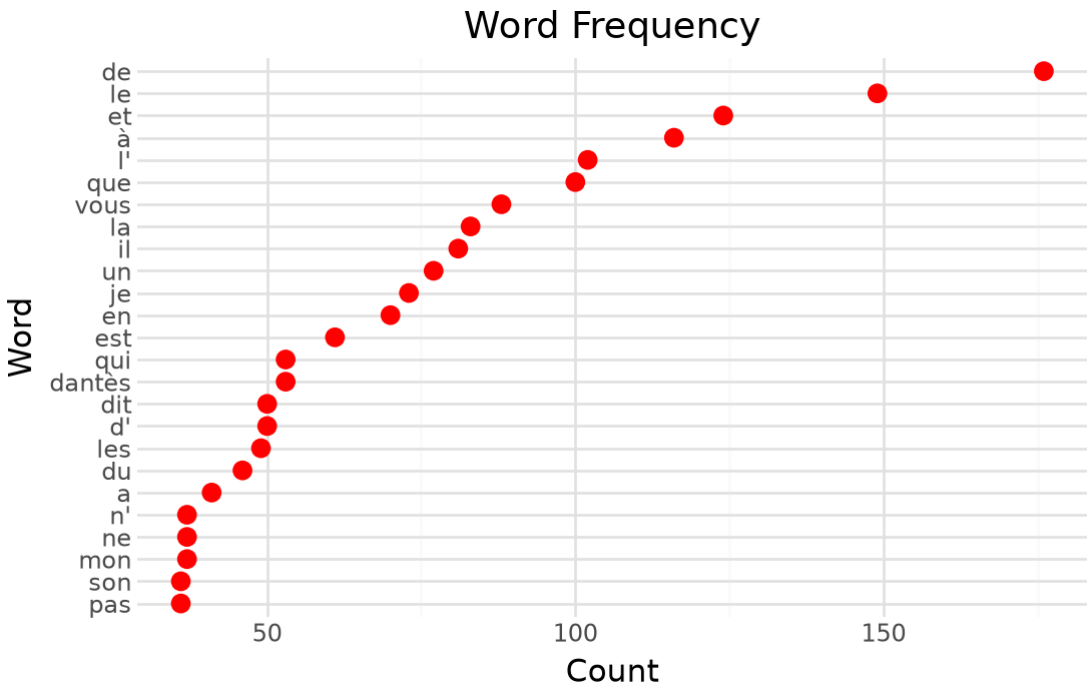
Nombre d'apparitions sur les 30 000 premiers caractères du Comte de Monte Cristo

Mot	Nombre d'occurrences	
ne	37	
n'	37	
mon	37	
son	36	
pas	36	

Nombre d'apparitions sur les 30 000 premiers caractères du Comte de Monte Cristo

Si on représente graphiquement ce classement

```
(
    ggplot(sorted_token_counts, aes(x='word', y='count')) +
    geom_point(stat='identity', size = 3, color = "red") +
    scale_x_discrete(
      limits=sorted_token_counts.sort_values("count")["word"].tolist()
    ) +
    coord_flip() +
    theme_minimal() +
    labs(title='Word Frequency', x='Word', y='Count')
)
```



Nous nous concentrerons ultérieurement sur ces mots-valises car il sera important d'en tenir compte pour les analyses approfondies de nos documents.

Nous avons pu, par ces décomptes de mots, avoir une première intutition de la nature de notre corpus. Néanmoins, une approche un peu plus visuelle serait pertinente pour avoir un peu plus d'intuitions. Les nuages de mots (*wordclouds*) sont des représentations graphiques assez pratiques pour visualiser les mots les plus fréquents, lorsqu'elles ne sont pas utilisées à tort et à travers. Les *wordclouds* sont très simples à implémenter en `Python` avec le module `Wordcloud`. Quelques paramètres de mise en forme permettent même d'ajuster la forme du nuage à une image.

Exercice 2 : Wordcloud

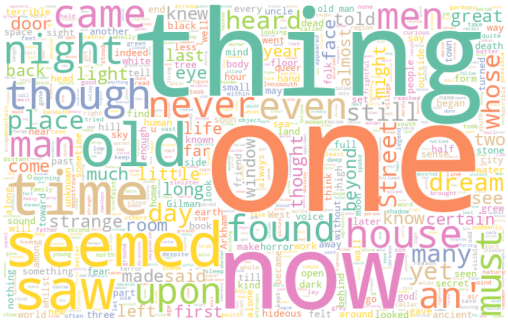
1. En utilisant la fonction `wordCloud`, faire trois nuages de mot pour représenter les mots les plus utilisés par chaque auteur du corpus `horror`[1].
2. Faire un nuage de mot du corpus `dumas` en utilisant un masque comme celui-ci

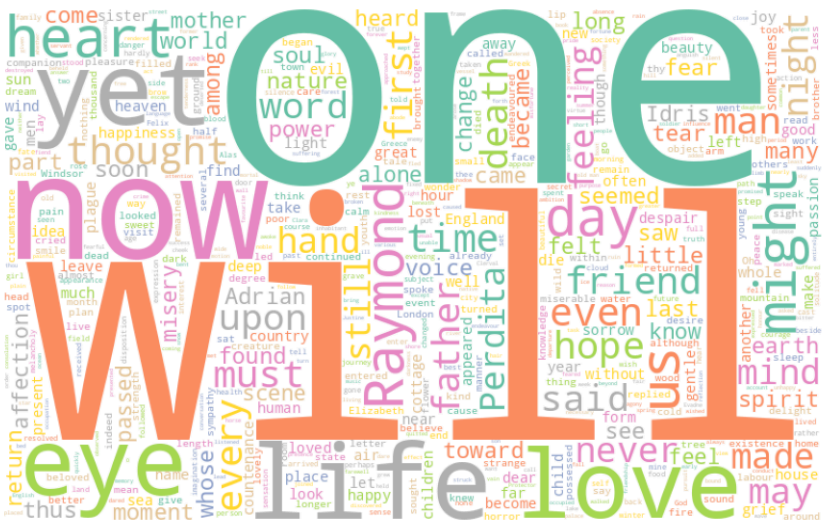
Exemple de masque pour la question 2



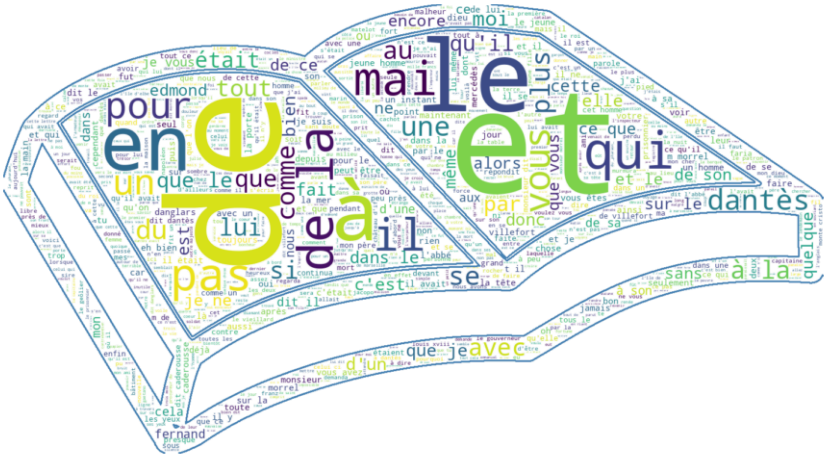
Figure 3. – URL de l'image: https://minio.lab.sspcloud.fr/lgaliana/generative-art/pythonds/book.png

Les nuages de points obtenus à la question 1 sont les suivants:


thing
one
now
upon
time
seemed
saw
night
though
never
even
old
man
found
house
came
heard
men
dream
street
whose
place
still
last
eye
life
thought
window
beyond
strange
room
day
long
little
much
made
said
first
left
three
knew
door
great
light
back
head
face
almost
might
year
hour
small
hand
mind
body
floor
dead
dark
open
hideous
felt
around
looked
ancient
terrible
world
sight
another
every
uncle
old
man
black
went
less
indeed
tell
while
hang
may
human
land
spa
full
began
dream
stone
city
water
sound
along
nothing
part
three
often
left
first
made
said
upon
saw
now
an
must
yet
many
secret
seen
open
dark
god
go
around
looked
ancient
beyond
thought
window
street
whose
two
dream
see
grew
later
know
certain
house
found
room
strange
day
long
little
much
come
past
sky
far
side
known
life
place
though
night
light
back
head
eye
last
indeed
mind
year
hour
small
hand
body
floor
dead
dark
door
great
face
almost
might
men
heard
came
knew
black
went
less



Alors que celui obtenu à partir de l'oeuvre de Dumas prend la forme



Si nous n'en étions pas convaincus, ces visualisations montrent clairement qu'il est nécessaire de nettoyer notre texte. Par exemple, en ce qui concerne l'oeuvre du Dumas, le nom du personnage principal, Dantès, est ainsi masqué par un certain nombre d'articles ou mots de liaison qui perturbent l'analyse. En ce qui concerne le corpus anglo-saxon, ce sont des termes similaires, comme *« the »*, *« of »*, etc.

Ces mots sont des *stop words*. Ceci est une démonstration par l'exemple qu'il vaut mieux nettoyer le texte avant de l'analyser (sauf si on est intéressé par la loi de Zipf, cf. exercice suivant).

3.4 Aparté: la loi de Zipf

Zipf, dans les années 1930, a remarqué une régularité statistique dans *Ulysse*, l'oeuvre de Joyce. Le mot le plus fréquent apparaissait x fois, le deuxième mot le plus fréquent 2 fois moins, le suivant 3 fois moins que le premier, etc. D'un point de vue statistique, cela signifie que la fréquence d'occurrence $f(n_i)$ d'un mot est liée à son rang n_i dans l'ordre des fréquences par une loi de la forme

$$f(n_i) = c/n_i$$

où c est une constante.

Plus généralement, on peut dériver la loi de Zipf d'une distribution exponentiellement décroissante des fréquences : $f(n_i) = cn_i^{-k}$. Sur le plan empirique, cela signifie qu'on peut utiliser les régressions poissonniennes pour estimer les paramètres de la loi, ce qui prend la spécification suivante

$$\mathbb{E}\Big(f(n_i)|n_i \Big) = \exp(\beta_0 + \beta_1 \log(n_i))$$

Les modèles linéaires généralisés (GLM) permettent de faire ce type de régression. En `Python`, ils sont disponibles par le biais du *package* `statsmodels`, dont les sorties sont très inspirées des logiciels payants spécialisés dans l'économétrie comme `Stata`.

```
count_words = pd.DataFrame({'counter' : horror
    .groupby('Author')
    .apply(lambda s: ' '.join(s['Text']).split())
    .apply(lambda s: Counter(s))
    .apply(lambda s: s.most_common())
    .explode()}
)
count_words[['word','count']]          =          pd.DataFrame(count_words['counter'].tolist(),
index=count_words.index)
count_words = count_words.reset_index()

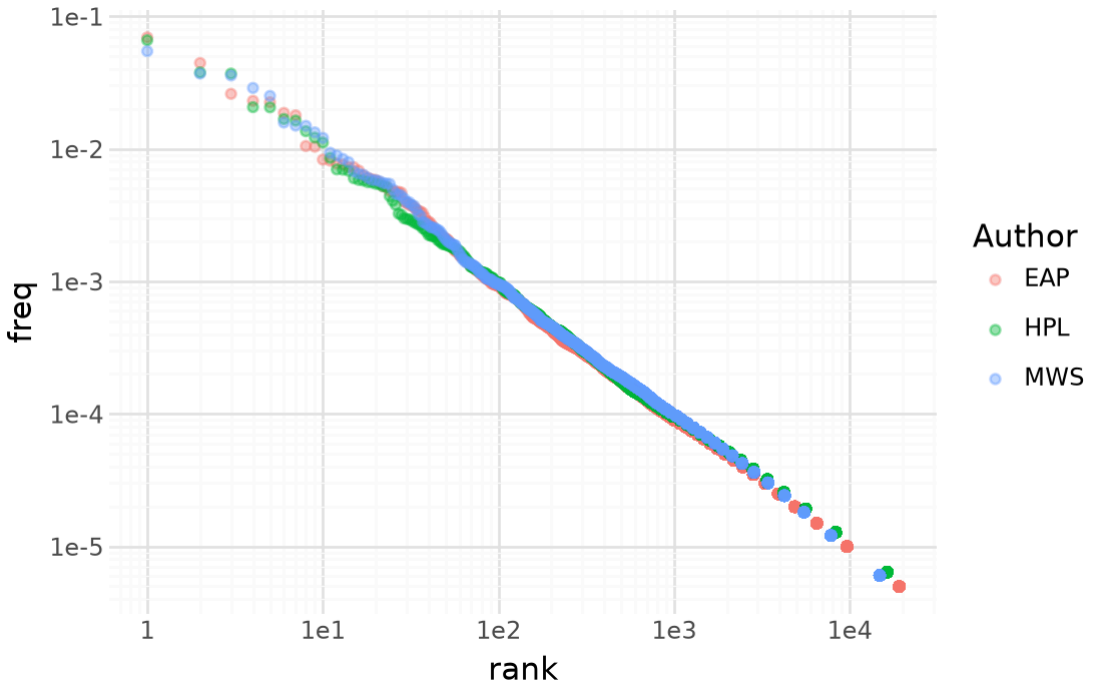
count_words = count_words.assign(
    tot_mots_auteur = lambda x: (x.groupby("Author")['count'].transform('sum')),
    freq = lambda x: x['count'] /  x['tot_mots_auteur'],
    rank = lambda x: x.groupby("Author")['count'].transform('rank', ascending = False)
)
```

Commençons par représenter la relation entre la fréquence et le rang:

```
from plotnine import *

g = (
    ggplot(count_words) +
    geom_point(aes(y = "freq", x = "rank", color = 'Author'), alpha = 0.4) +
    scale_x_log10() + scale_y_log10() +
    theme_minimal()
)
```

Nous avons bien, graphiquement, une relation log-linéaire entre les deux :



Avec `statsmodels`, vérifions plus formellement cette relation:

```
import statsmodels.api as sm
import numpy as np

exog = sm.add_constant(np.log(count_words['rank'].astype(float)))

model = sm.GLM(count_words['freq'].astype(float), exog, family = sm.families.Poisson()).fit()

# Afficher les résultats du modèle
print(model.summary())
```

```
                 Generalized Linear Model Regression Results
==============================================================================
Dep. Variable:                   freq   No. Observations:                69301
Model:                            GLM   Df Residuals:                    69299
Model Family:                 Poisson   Df Model:                            1
Link Function:                    Log   Scale:                          1.0000
Method:                          IRLS   Log-Likelihood:                -23.011
Date:                Tue, 20 Oct 2026   Deviance:                     0.065676
Time:                        06:44:55   Pearson chi2:                   0.0656
No. Iterations:                     5   Pseudo R-squ. (CS):          0.0002431
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
const         -2.4388      1.089     -2.239      0.025      -4.574      -0.303
rank          -0.9831      0.189     -5.196      0.000      -1.354      -0.612
==============================================================================
```

Le coefficient de la régression est presque 1 ce qui suggère bien une relation quasiment log-linéaire entre le rang et la fréquence d'occurrence d'un mot. Dit autrement, le mot le plus utilisé l'est deux fois plus que le deuxième mot le plus fréquent qui l'est trois plus que le troisième, etc. On retrouve bien empiriquement cette loi sur ce corpus de trois auteurs.

4 Nettoyage de textes

4.1 Retirer les *stop words*

Nous l'avons vu, que ce soit en Français ou Anglais, un certain nombre de mots de liaisons, nécessaires sur le plan grammatical mais peu porteur d'information, nous empêchent de saisir les principaux mots vecteurs d'information dans notre corpus.

Il est donc nécessaire de nettoyer notre corpus en retirant ces termes. Ce travail de nettoyage va d'ailleurs au-delà d'un simple retrait de mots. C'est également l'occasion de retirer d'autres sèmes gênants, par exemple la ponctuation.

Commençons par télécharger le corpus de *stopwords*

```
import nltk
nltk.download('stopwords')
```

```
[nltk_data] Downloading package stopwords to /github/home/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
```

```
True
```

La liste des *stopwords* anglais dans `NLTK` est la suivante:

```
from nltk.corpus import stopwords
", ".join(stopwords.words("english"))
```

```
"a, about, above, after, again, against, ain, all, am, an, and, any, are, aren, aren't, as,
at, be, because, been, before, being, below, between, both, but, by, can, couldn, couldn't,
d, did, didn, didn't, do, does, doesn, doesn't, doing, don, don't, down, during, each, few,
for, from, further, had, hadn, hadn't, has, hasn, hasn't, have, haven, haven't, having, he,
he'd, he'll, her, here, hers, herself, he's, him, himself, his, how, i, i'd, if, i'll, i'm,
in, into, is, isn, isn't, it, it'd, it'll, it's, its, itself, i've, just, ll, m, ma, me,
mightn, mightn't, more, most, mustn, mustn't, my, myself, needn, needn't, no, nor, not, now, o,
of, off, on, once, only, or, other, our, ours, ourselves, out, over, own, re, s, same, shan,
shan't, she, she'd, she'll, she's, should, shouldn, shouldn't, should've, so, some, such, t,
than, that, that'll, the, their, theirs, them, themselves, then, there, these, they, they'd,
they'll, they're, they've, this, those, through, to, too, under, until, up, ve, very, was, wasn,
wasn't, we, we'd, we'll, we're, were, weren, weren't, we've, what, when, where, which, while,
who, whom, why, will, with, won, won't, wouldn, wouldn't, y, you, you'd, you'll, your, you're,
yours, yourself, yourselves, you've"
```

Celle de `SpaCy` est plus riche (nous avons déjà téléchargé le corpus `en_core_web_sm` en question):

```
nlp_english = spacy.load('en_core_web_sm')
stop_words_english = nlp_english.Defaults.stop_words
", ".join(stop_words_english)
```

```
"never, hereby, that, must, why, indeed, where, 'll, seems, while, whether, beforehand, give,
someone, forty, move, myself, thereby, empty, whereas, any, no, along, part, three, together,
're, did, sixty, sometime, or, upon, so, eight, therein, still, after, when, you, being, put,
their, been, only, we, yours, doing, 're, mine, often, who, its, could, because, several, 'm,
were, i, get, beyond, whereby, an, various, regarding, should, through, within, under, nowhere,
'll, therefore, amongst, be, each, those, another, front, against, until, afterwards, not, fifty,
```

```
your, meanwhile, once, noone, though, for, becomes, 've, above, twenty, become, something,
hereupon, whole, four, does, except, becoming, as, thus, and, however, but, between, 'm, off,
down, hereafter, cannot, using, make, me, nor, alone, 've, onto, became, ca, ten, first, us, done,
n't, elsewhere, all, take, are, whose, due, her, my, really, will, of, twelve, seeming, 're,
ever, 'm, to, already, please, ours, amount, mostly, thereafter, moreover, hence, throughout,
there, others, was, fifteen, 's, either, latter, third, last, him, thereupon, five, used, in,
six, anyhow, none, thru, whereafter, very, seemed, too, among, enough, call, nobody, hers, yet,
somewhere, two, many, almost, least, few, whoever, the, made, since, even, perhaps, behind,
itself, into, anything, herself, have, side, serious, it, other, 've, here, latterly, per, same,
from, had, bottom, thence, before, am, both, she, sometimes, everything, toward, quite, always,
just, wherein, without, neither, keep, this, every, 'd, else, with, say, everyone, go, wherever,
is, what, our, show, ourselves, out, everywhere, 'll, nine, yourself, own, whereupon, much, most,
besides, across, than, eleven, about, then, himself, on, herein, n't, anyone, name, anyway, one,
namely, more, they, well, 'd, somehow, them, has, formerly, n't, how, via, 's, below, rather,
otherwise, whenever, full, by, hundred, also, over, seem, further, 's, do, he, top, again,
which, see, whence, themselves, former, his, back, around, can, less, may, nothing, these, up,
anywhere, some, whom, towards, although, a, whither, if, such, beside, might, 'd, next, whatever,
nevertheless, at, unless, re, during, yourselves, would, now"
```

Si cette fois on prend la liste des *stopwords* français dans `NLTK`:

```
", ".join(stopwords.words("french"))
```

```
'au, aux, avec, ce, ces, dans, de, des, du, elle, en, et, eux, il, ils, je, la, le, les, leur,
lui, ma, mais, me, même, mes, moi, mon, ne, nos, notre, nous, on, ou, par, pas, pour, qu, que,
qui, sa, se, ses, son, sur, ta, te, tes, toi, ton, tu, un, une, vos, votre, vous, c, d, j, l, à,
m, n, s, t, y, été, étée, étées, étés, étant, étante, étants, étantes, suis, es, est, sommes,
êtes, sont, serai, seras, sera, serons, serez, seront, serais, serait, serions, seriez, seraient,
étais, était, étions, étiez, étaient, fus, fut, fûmes, fûtes, furent, sois, soit, soyons, soyez,
soient, fusse, fusses, fût, fussions, fussiez, fussent, ayant, ayante, ayantes, ayants, eu, eue,
eues, eus, ai, as, avons, avez, ont, aurai, auras, aura, aurons, aurez, auront, aurais, aurait,
aurions, auriez, auraient, avais, avait, avions, aviez, avaient, eut, eûmes, eûtes, eurent, aie,
aies, ait, ayons, ayez, aient, eusse, eusses, eût, eussions, eussiez, eussent'
```

On voit que celle-ci n'est pas très riche et mériterait d'être plus complète. Celle de `SpaCy` correspond mieux à ce qu'on attend

```
stop_words_french = nlp.Defaults.stop_words
", ".join(stop_words_french)
```

```
"dix, voici, ouvert, quelconque, laisser, autre, malgré, suivant, lors, elle-même, façon,
cinquante, s', chez, retour, nous-mêmes, vais, va, devant, pendant, siens, lui-même, plusieurs,
desormais, déjà, gens, dix-neuf, quatre-vingt, soi-meme, cinq, es, suivantes, nôtres, seraient,
également, differents, quatrièmement, o, maintenant, aura, tant, vas, meme, t', plutôt, celles-
la, certes, tel, rend, lès, or, de, n', pourrais, importe, entre, ouverte, qu', dix-huit, votre,
devra, relativement, quoique, nôtre, plutot, cinquantième, deuxième, doivent, sous, na, peuvent,
pouvait, tente, tres, laquelle, vôtre, telle, effet, rendre, dehors, different, semblable,
anterieures, tels, etc, vos, celle, moi, nouveau, i, ouias, seule, différent, ils, huitième,
hé, lesquels, semble, hem, nos, plus, différentes, telles, cela, parce, dès, dedans, mienne,
ouverts, anterieure, hue, l', tous, là, durant, près, ai, pas, deuxièmement, faisaient, ayant,
mien, celle-là, ah, dix-sept, selon, apres, tenant, chacun, étais, derrière, precisement, jusque,
puisque, nul, memes, cent, sait, septième, parler, elles-memes, deux, hors, fait, hui, parle,
environ, précisement, du, bat, differente, enfin, concernant, ton, quatrième, lequel, moins,
pense, souvent, suivants, delà, vous, m', as, ès, onzième, etais, l', premièrement, avec, comme,
soit, soixante, restant, dire, sera, merci, vé, allons, celles, sien, mais, me, autres, moi-
meme, se, encore, peut, directe, c', ci, à, celui-la, avais, déja, quelles, longtemps, â,
```

```
vous-mêmes, tend, nombreux, miennes, différents, suis, suit, ha, c', facon, exactement, aupres,
revoilà, serait, auxquelles, elle-meme, da, chaque, tienne, dit, cinquième, compris, suivre,
ni, celles-là, houp, seuls, etaient, mes, ce, abord, celles-ci, desquelles, ceci, et, être,
revoici, ait, peux, vers, quelle, lesquelles, quarante, seul, autrui, neanmoins, ainsi, est,
d', qui, voila, étaient, reste, sept, ont, toi-même, vu, mêmes, sinon, eux-mêmes, six, tiennes,
premier, avons, mon, tout, seize, quel, son, allaient, maint, permet, néanmoins, autrement,
spécifiques, votres, quant-à-soi, ses, pu, aie, antérieure, aurait, il, mille, treize, diverses,
envers, basee, sa, les, afin, cette, elles-mêmes, ta, ça, première, eux, eh, si, seront, très,
ouste, trente, quatorze, dits, pourquoi, étant, leur, notamment, auquel, ho, spécifique, puis,
etant, combien, un, antérieur, possible, celui-là, lui, moindres, tien, celui, partant, huit,
neuvième, aussi, quatre, stop, doit, parlent, dont, hep, vôtres, assez, antérieures, m', derriere,
tenir, anterieur, devers, deja, attendu, préalable, té, désormais, au, debout, auront, miens,
était, douzième, personne, j', semblent, sienne, troisièmement, trois, excepté, donc, notre,
aux, toi-meme, toujours, toi, directement, ces, bas, tellement, certaines, tiens, touchant, t',
nombreuses, je, hou, quant, suffisante, quiconque, outre, sont, elle, même, depuis, cet, jusqu,
uns, quand, suffisant, pourrait, desquels, ceux, par, comment, certaine, celle-la, dans, voilà,
restent, n', parfois, auraient, diverse, duquel, sur, juste, des, sauf, surtout, differentes,
on, faisant, troisième, une, moi-même, ne, toute, j', divers, semblaient, tes, eu, feront, pour,
ceux-ci, douze, relative, certains, certain, dite, egalement, parmi, via, ma, prealable, seules,
specifiques, où, fais, sent, sixième, auxquels, avaient, suivante, quinze, d', lui-meme, siennes,
avait, peu, toutes, onze, lorsque, en, leurs, celui-ci, dessus, dessous, après, seulement, unes,
etait, procedant, suffit, pres, car, quelque, nous, hormis, s', la, cependant, soi-même, ô,
revoila, quelqu'un, quelques, différente, ou, soi, possibles, proche, sans, te, font, etre, qu',
avoir, malgre, cinquantaine, specifique, tu, chacune, elles, a, vont, hi, y, quels, le, vingt,
alors, avant, celle-ci, que, dixième, déjà, ceux-là, quoi"
```

Exercice 3 : Nettoyage du texte

1. Reprendre l'ouvrage de Dumas et nettoyer celui-ci avec `Spacy`. Refaire le nuage de mots et conclure.
2. Faire ce même exercice sur le jeu de données anglo-saxon. Idéalement, vous devriez être en mesure d'utiliser la fonctionnalité de *pipeline* de `SpaCy`.

```python
# Function to clean the text
def clean_text(doc):
    # Tokenize, remove stop words and punctuation, and lemmatize
    cleaned_tokens = [
        token.lemma_ for token in doc if not token.is_stop and not token.is_punct
    ]
    # Join tokens back into a single string
    cleaned_text = " ".join(cleaned_tokens)
    return cleaned_text
```

Ces retraitements commencent à porter leurs fruits puisque des mots ayant plus de sens commencent à se dégager, notamment les noms des personnages (Dantès, Danglart, etc.):

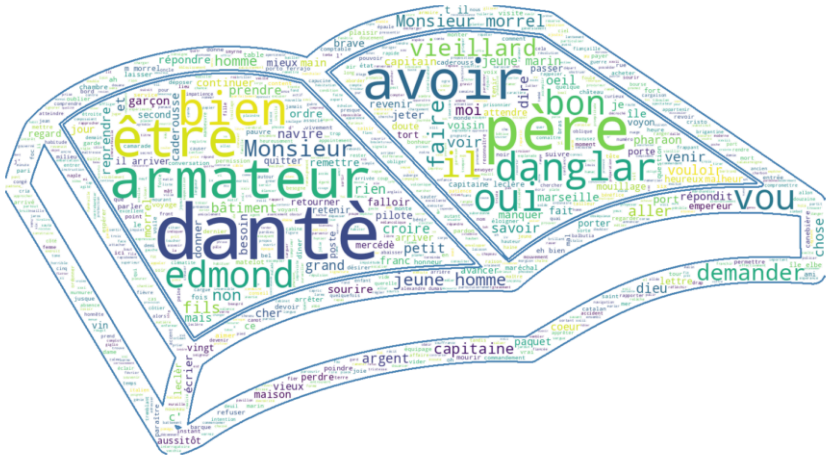


Figure 8. – Nuage de mot produit à partir du Comte de Monte Cristo après nettoyage



Figure 9. – Lovercraft


find
say
time
man
know
little
thing
take
eye
long
word
come
mean
speak
see
feel
day
hand
matter
look
fact
life
love
great
good
make
give
hear
leave
place
length
point
moment
minute
character
doubt
night
room
mind
heart
head
person
body
foot
think
pass
course
large
water
hour
friend
form
appear
way
house
true
high
throw
go
reason
remain
open
stand
believe
question
tell
attention
fall
end
hand
manner
suppose
understand
object
discover
tree
half
ground
proceed
far
wild
sure
second
figure
heart
sound
lose
bear
air
life
thought
voice
possible
bring
subject
regard
merely
attempt
direction
work
sit
Mr
power
fail
low
tone
sea
spirit
difficulty
idea
let
grow
oh
arise
close
consider
turn
reply
light
enter
natural
mere
endeavor
hold
table
old
position
account
observe
world
wall
window
nearly
case
smallview
human
earth
purpose
remember
degree
Mr
single
lead
stand
believe
direction
question
tell
attention
sit
room
open
minute
character
doubt
night
great
good
go
moment
point
length
place
fall
call
kind
soul
ordinary
principle
thought
bring
possible

> **Note**
>
> Pour disposer du corpus nécessaire à la lemmatisation, il faut, la première fois, télécharger celui-ci grâce aux commandes suivantes :

```
import nltk
nltk.download('wordnet')
nltk.download('omw-1.4')
```

Prenons cette chaine de caractère,

```
"naples. comme d'habitude, un pilote côtier partit aussitôt du port, rasa le château d'if, et
alla aborder le navire entre le cap de morgion et l'île de rion. aussitôt, co"
```

La version racinisée est la suivante:

```
"napl,.,comm,d'habitud,,,un,pilot,côti,part,aussitôt,du,port,,,ras,le,château,d'if,,,et,alla,abord,le,navir,entre,le,
```

A ce niveau, les mots commencent à être moins intelligibles par un humain mais peuvent rester intelligible pour la machine. Ce choix n'est néanmoins pas neutre et sa pertinence dépend du cas d'usage.

Les lemmatiseurs permettront des harmonisations plus subtiles. Ils s'appuient sur des bases de connaissance, par exemple *WordNet*, une base lexicographique ouverte. Par exemple, les mots « *women* », « *daughters* » et « *leaves* » seront ainsi lemmatisés de la manière suivante :

```
from nltk.stem import WordNetLemmatizer
lemm = WordNetLemmatizer()

for word in ["women","daughters", "leaves"]:
    print(f"The lemmatized form of {word} is: {lemm.lemmatize(word)}")
```

```
The lemmatized form of women is: woman
The lemmatized form of daughters is: daughter
The lemmatized form of leaves is: leaf
```

> **Exercice 4 : Lemmatisation avec nltk**
>
> Sur le modèle précédent, utiliser un `WordNetLemmatizer` sur le corpus `dumas[1030:1200]` et observer le résultat.

La version lemmatisée de ce petit morceau de l'oeuvre de Dumas est la suivante:

```
"naples, ., comme, d'habitude, ,, un, pilote, côtier, partit, aussitôt, du, port, ,, rasa, le,
château, d'if, ,, et, alla, aborder, le, navire, entre, le, cap, de, morgion, et, l'île, de,
rion, ., aussitôt, ,, co"
```

4.3 Limite

Dans les approches fréquentistes, où on recherche la proximité entre des textes par la co-occurrence de termes, cette question de la création de classes d'équivalence est fondamentale. Les mots sont identiques ou différents, il n'y a pas de différence subtile entre eux. Par exemple, on devra soit déclarer

que *« python »* et *« pythons »* sont équivalents, soient qu'ils sont différents, sans différence de degré entre *« pythons »*, *« anaconda »* ou *« table »* par rapport à *« python »*. Les approches modernes, non plus exclusivement basées sur des fréquences d'apparition, permettront d'introduire de la subtilité dans la synthétisation de l'information présente dans les données textuelles.

Informations additionnelles

i Environnement Python

Ce site a été construit automatiquement par le biais d'une action `Github` utilisant le logiciel de publication reproductible `Quarto` (version 1.10.18).

L'environnement utilisé pour obtenir les résultats est reproductible par le biais d'uv. Le fichier `pyproject.toml` utilisé pour construire cet environnement est disponible sur le dépôt `linogaliana/python-datascientist`

```
[project]
name = "python-datascientist"
version = "0.1.0"
description = "Source code for Lino Galiana's Python for data science course"
readme = "README.md"
requires-python = "≥3.13,<3.14"
dependencies = [
    "altair≥6.0.0",
    "cartiflette≥0.2",
    "contextily==1.6.2",
    "duckdb≥0.10.1",
    "folium≥0.19.6",
    "gdal==3.11.4",
    "graphviz==0.20.3",
    "great-tables≥0.12.0",
    "gt-extras≥0.0.8",
    "ipykernel≥6.29.5",
    "jupyter≥1.1.1",
    "jupyter-cache≥1.0.0",
    "kaleido≥0.2.1",
    "langchain-community≥0.3.27",
    "loguru==0.7.3",
    "markdown≥3.8",
    "nbclient≥0.10.0",
    "nbformat≥5.10.4",
    "nltk≥3.9.1",
    "openpyxl≥3.1.5",
    "pandas≥3.0",
    "pip≥25.1.1",
    "plotly≥6.1.2",
    "plotnine≥0.15",
    "polars≥1.8.2",
    "py7zr≥1.1.3",
    "pyarrow≥17.0.0",
    "pynsee≥0.1.8",
    "python-dotenv≥1.0.1",
    "python-frontmatter≥1.1.0",
    "pywaffle≥1.1.1",
    "requests≥2.32.3",
    "scikit-image≥0.24.0",
```

```
    "scikit-learn≥1.8.0",
    "scipy≥1.13.0",
    "seaborn≥0.13.2",
    "selenium<4.39.0",
    "spacy≥3.8.4",
    "webdriver-manager≥4.0.2",
    "wordcloud==1.9.3",
]

[tool.uv.sources]
gdal = [
  { index = "gdal-wheels", marker = "sys_platform == 'linux'" },
  { index = "geospatial_wheels", marker = "sys_platform == 'win32'" },
]

[[tool.uv.index]]
name = "geospatial_wheels"
url = "https://nathanjmcdougall.github.io/geospatial-wheels-index/"
explicit = true

[[tool.uv.index]]
name = "gdal-wheels"
url = "https://gitlab.com/api/v4/projects/61637378/packages/pypi/simple"
explicit = true

[dependency-groups]
dev = [
    "nb-clean≥4.0.1",
]
```

Pour utiliser exactement le même environnement (version de `Python` et *packages*), se reporter à la documentation d'`uv`.

Bibliographie